

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	A.Anusha	Reg. No.:	192472083
Section / Batch:	CSE – AI / 2024	Date:	05-08-26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Q3. Case Study: Search Engine Query Processing (Binary Search)

A search engine maintains a sorted index of keywords and must retrieve results quickly. Analyze how Binary Search improves search efficiency. Identify key requirements such as sorted data and fast retrieval. Apply divide and conquer principles to explain the process. Analyze time complexity and performance. Design a solution and justify why Binary Search is optimal in this context.

Code of Conduct

I A. Anusha/192472083(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A.Anusha

Case Study Report

Title: Search Engine Query Processing Using Binary Search

1. Description of the Real-World Problem

Search engines such as Google, Bing, and Yahoo process millions of search queries every day. To provide results quickly, they maintain a sorted index of keywords. When a user enters a keyword, the search engine must locate it as fast as possible. Searching each keyword one by one would take too much time for large datasets. Therefore, an efficient searching algorithm is required to ensure quick retrieval of information.

2. Problem Statement

A search engine stores its keywords in sorted order. When a user enters a search query, the system must determine whether the keyword exists in the index and retrieve the corresponding results with minimum search time. The objective is to design an efficient searching solution using the Binary Search algorithm.

3. Algorithm Used

Binary Search Algorithm

Algorithm:

1. Store all keywords in sorted order.
2. Initialize:
 - $low = 0$
 - $high = n - 1$
3. Find the middle element:
 - $mid = (low + high) / 2$
4. Compare the search keyword with the middle element.
5. If both are equal, the keyword is found.
6. If the search keyword is smaller, search the left half ($high = mid - 1$).
7. If the search keyword is larger, search the right half ($low = mid + 1$).
8. Repeat Steps 3–7 until the keyword is found or $low > high$.
9. Display whether the keyword is found or not.

4. Justification for Choosing the Algorithm (Why This Algorithm?)

Binary Search is the best choice because:

- It works efficiently on sorted data.
- It uses the Divide and Conquer approach.
- It reduces the search space by half after every comparison.
- It requires very few comparisons even for large datasets.
- It provides much faster searching than Linear Search.
- It is widely used in search engines, databases, and indexing systems.

For example, searching among 1,000,000 keywords requires only about 20 comparisons using Binary Search.

5. Complexity Analysis

Table 1: Complexity Analysis

Case	Time Complexity	Explanation
Best Case	$O(1)$	The search key is found in the first comparison (middle element).
Average Case	$O(\log n)$	The search space is repeatedly divided into two halves until the element is found.
Worst Case	$O(\log n)$	The search continues until the last possible comparison or the element is not found.
Space Complexity	$O(1)$ (Iterative)	Only a few variables (low, high, and mid) are used, so constant memory is required.

Table 2: Maximum Comparisons in Binary Search

Number of Elements (n)	Maximum Comparisons ($\log_2 n$)
8	3
16	4
32	5
64	6
128	7
256	8
512	9
1,024	10
1,000,000	≈ 20

Analysis

Binary Search divides the search space into two equal halves after every comparison. Therefore, the number of comparisons grows logarithmically with the input size, making it highly efficient for large sorted datasets.

6. Manual Execution (Step-by-Step Process)

Sorted Keywords

AI, Algorithm, Binary, Cloud, Database, Machine, Network, Python, Search

Search Keyword: Machine

Step 1

- Low = 0
- High = 8
- Mid = 4
- Middle Element = Database

Machine > Database

Search the right half.

Step 2

Remaining Elements:

Machine, Network, Python, Search

- Low = 5
- High = 8
- Mid = 6
- Middle Element = Network

Machine < Network

Search the left half.

Step 3

Remaining Element:

Machine

The keyword matches the middle element.

Result: Keyword Found

Total Comparisons: 3

7. Applications

Binary Search is commonly used in:

- Search engine keyword lookup
- Database indexing
- Online product search
- Dictionary word search
- Phone contact search
- Library management systems
- Student record management
- File searching
- Operating systems

8. Advantages

- Fast searching technique.
- Reduces search space by half after every iteration.
- Requires fewer comparisons.
- Highly efficient for large datasets.
- Easy to implement.
- Uses the Divide and Conquer strategy.
- Improves overall search performance.

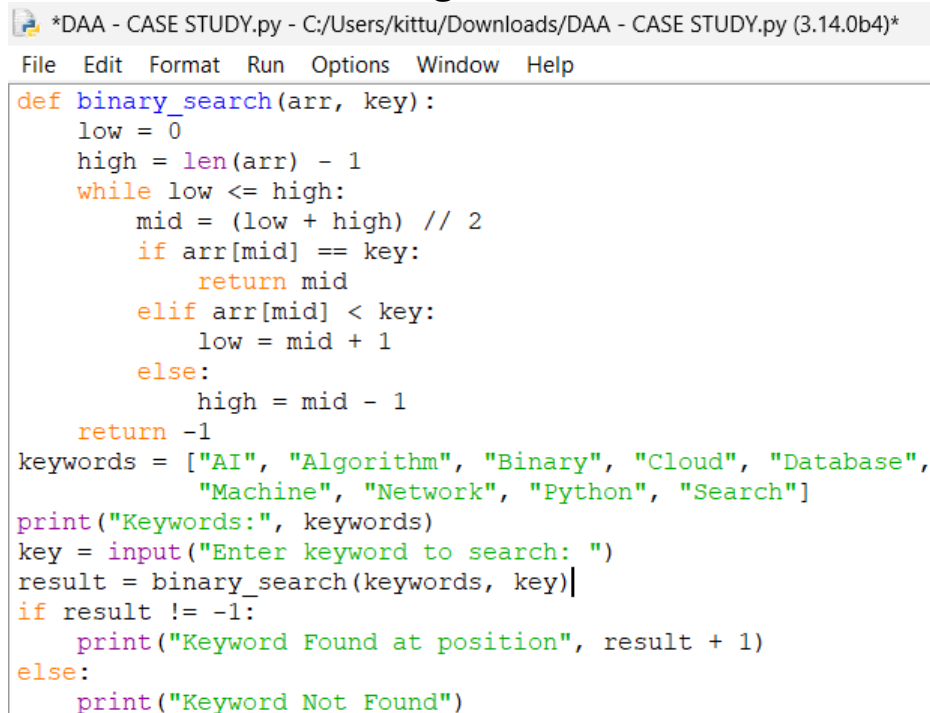
9. Limitations

- Works only on sorted data.
- Sorting requires additional processing.
- Not suitable for unsorted datasets.
- Maintaining sorted order can be costly when data changes frequently.

10. Comparison with Other Algorithms

Feature	Linear Search	Binary Search
Data Requirement	Works on sorted or unsorted data	Works only on sorted data
Searching Method	Sequential Search	Divide and Conquer
Best Case Time Complexity	O(1)	O(1)
Average Case Time Complexity	O(n)	O(log n)
Worst Case Time Complexity	O(n)	O(log n)
Number of Comparisons	More comparisons	Fewer comparisons
Suitable for Large Datasets	No	Yes
Speed	Slower	Faster
Space Complexity (Iterative)	O(1)	O(1)
Main Advantage	Simple and works on any dataset	Very fast for sorted datasets
Main Limitation	Inefficient for large datasets	Requires data to be sorted

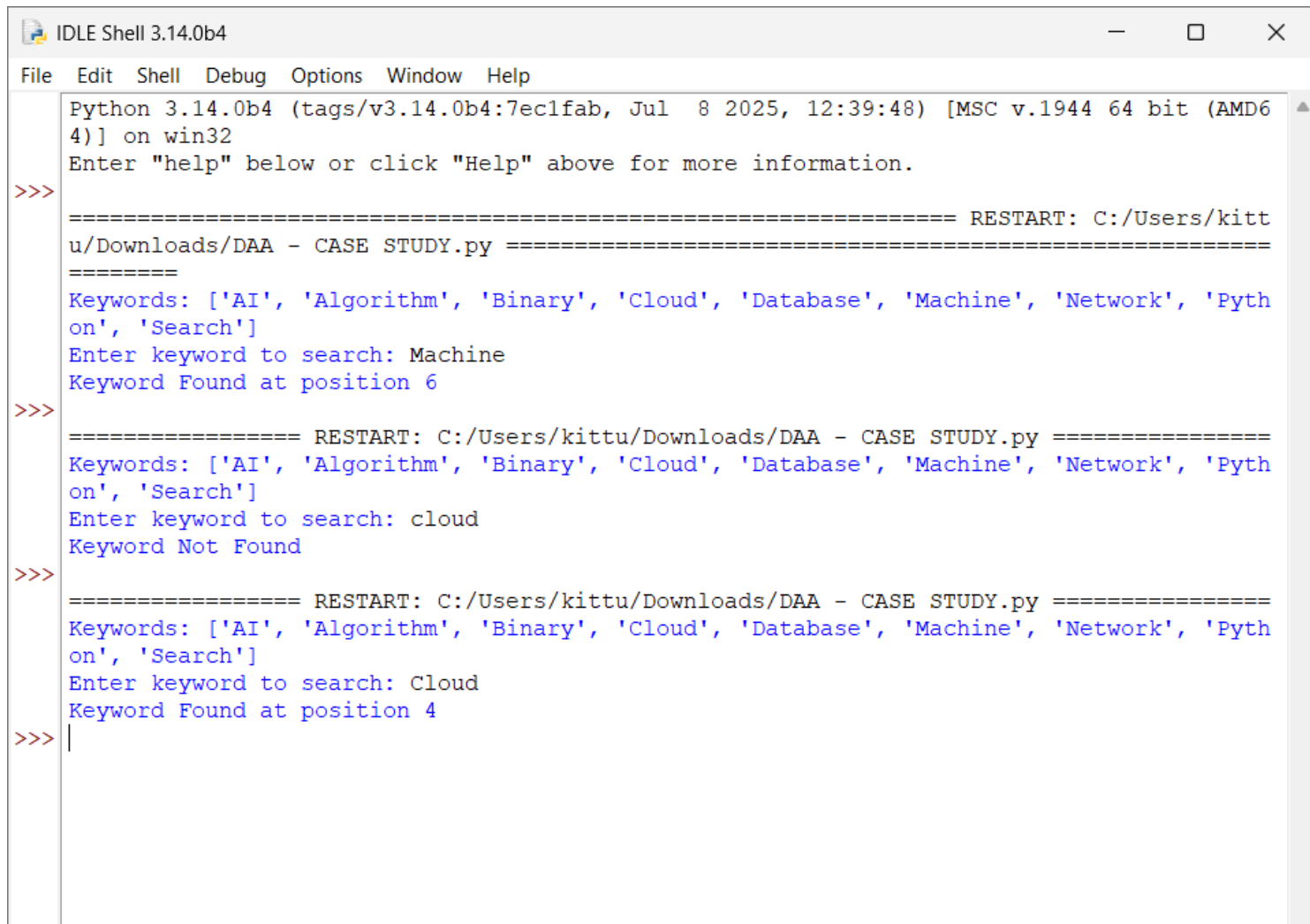
11. Screenshot of the Program



```
*DAA - CASE STUDY.py - C:/Users/kittu/Downloads/DAA - CASE STUDY.py (3.14.0b4)*
File Edit Format Run Options Window Help

def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1
keywords = ["AI", "Algorithm", "Binary", "Cloud", "Database",
            "Machine", "Network", "Python", "Search"]
print("Keywords:", keywords)
key = input("Enter keyword to search: ")
result = binary_search(keywords, key)
if result != -1:
    print("Keyword Found at position", result + 1)
else:
    print("Keyword Not Found")
```

12. Output Screenshot



```
IDLE Shell 3.14.0b4
File Edit Shell Debug Options Window Help
Python 3.14.0b4 (tags/v3.14.0b4:7ec1fab, Jul  8 2025, 12:39:48) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/kittu/Downloads/DAA - CASE STUDY.py =====
Keywords: ['AI', 'Algorithm', 'Binary', 'Cloud', 'Database', 'Machine', 'Network', 'Python', 'Search']
Enter keyword to search: Machine
Keyword Found at position 6
>>>
===== RESTART: C:/Users/kittu/Downloads/DAA - CASE STUDY.py =====
Keywords: ['AI', 'Algorithm', 'Binary', 'Cloud', 'Database', 'Machine', 'Network', 'Python', 'Search']
Enter keyword to search: cloud
Keyword Not Found
>>>
===== RESTART: C:/Users/kittu/Downloads/DAA - CASE STUDY.py =====
Keywords: ['AI', 'Algorithm', 'Binary', 'Cloud', 'Database', 'Machine', 'Network', 'Python', 'Search']
Enter keyword to search: Cloud
Keyword Found at position 4
>>> |
```

13. Conclusion

The Binary Search algorithm is an efficient searching technique for sorted datasets. It follows the Divide and Conquer approach by repeatedly dividing the search space into two halves, which significantly reduces the number of comparisons. In this case study, Binary Search successfully located the required keyword in the search engine index with a time complexity of $O(\log n)$. Compared to Linear Search, it provides much faster retrieval for large datasets, making it an ideal choice for search engine query processing, database indexing, and other applications that require quick and efficient searching.

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____